Optimizing Software Testing Effectiveness

Divya Ranjan, Ph.D.

Software Testing Program Manager

Cognex Corporation, MA

Optimization

- Limited resources
  - Design/strategize -> control -> allocate
  - So that you maximize/get best possible results

- Minimize resources and maximize your results.
- Suck every single bit out of it
- How would you do it -> by strategy/designing

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Scope

- Optimizing- improving coverage, accuracy, efficiency, and alignment with business risk
- Security testing
- Effectiveness
- How to achieve this
  - What is the structured & systematic & reliable & sustainable approach to achieve that

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Types of testing

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Real world challenges in optimizing security testing effectiveness

- Tooling
  - High false-positive rates reducing trust in tools
  - Poor UI and navigation slow down analysis and remediation. Poor usability increases time to triage and remediate
  - Too much customization needed. Excessive manual processing required
  - Tools do not deliver the right information:
    - To the right people
    - At the right time
    - In the right context
    - Need extra processing. More automation.
  - Manual effort required to interpret and re-process results
  - Need for better/more automation and actionable insights
  - Difficulty selecting vendors that meet real operational needs
  - Vendor marketing claims often misalign with real-world effectiveness. Vendor selection driven by marketing, not outcomes. Lack of contextual, role-based insights
  - Findings are not mapped to real attack paths or exploitability or priority. Incorrect severity mapping.
  - Limited ability to customize scans for architecture and tech stack
  - Poor integration with:
    - Issue trackers (Jira, ADO)
    - Developer workflows
    - Cloud and container platforms
  - Results are security-centric, not developer-friendly
  - Reporting optimized for audits, not remediation and per your business needs and audiences
  - Scalability issues as environments and pipelines grow

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Real world challenges in optimizing security testing effectiveness

- Automation
  - CI/CD

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Real world challenges in optimizing security testing effectiveness

- Training
- Process improvement

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness

Risk-Based Prioritization

- Not all vulnerabilities deserve equal effort
- Prioritize testing based on:
  - Business impact
  - Data sensitivity (PII, regulated data)
  - Customer and attacker exposure
- Focus deepest testing on critical flows:
  - Authentication & authorization
  - Payments
  - High-risk APIs
- Choose your battles deliberately

Shift-Left Testing in the SDLC

- Move security testing earlier:
  - Requirements
  - Architecture
  - Design
- Catching defects early:
  - Reduces cost of fixes
  - Prevents rework
  - Improves delivery speed
- Embed security into planning—not just validation

Threat-Model-Driven Testing

- Use threat modeling to guide test depth and scope
- Identify:
  - Likely attack vectors
  - Abuse and misuse cases
- Apply structured frameworks:
  - STRIDE
  - MITRE ATT&CK
- Test what attackers are most likely to exploit

Layered Testing Strategy

- No single tool provides complete coverage
- Combine complementary approaches:
  - SAST + DAST + IAST
  - Manual + automated testing
  - Application + API + cloud + infrastructure testing
- Layered controls reduce blind spots and false confidence

Smart Tool Placement & DevSecOps Integration

- Place tools intentionally in the pipeline:
  - SAST → Commit / CI
  - SCA → Build pipelines
  - DAST → Staging
  - Pen testing → High-risk releases
- Use incremental and differential scans
- Give developers fast, actionable feedback

Noise Reduction & Signal Quality

- Optimize effectiveness by reducing friction:
  - Tune rules and thresholds
  - Deduplicate findings across tools
  - Baseline accepted risks
- Focus on exploitable, high-confidence issues
- Build developer trust in security results

Depth Through Adversarial & Logic Testing

- Go beyond CVEs and automated scans
- Test:
  - Business logic abuse
  - Privilege escalation
  - Chained vulnerabilities
- Use:
  - Red team / adversarial testing
  - Exploratory security testing
- Simulate real attacker behavior

Automation with Purpose (Not Volume)

- Automate what benefits from scale:
  - Regression security tests
  - API tests
  - High-volume repeatable checks
- Avoid automating unstable or low-value areas
- Keep manual testing for:
  - Logic flaws
  - Complex abuse cases

Continuous Measurement & Improvement

- Optimize using feedback loops, not one-time tests
- Measure effectiveness with outcome-based metrics:
  - Critical issues found pre-production
  - Mean time to remediate high-risk findings
  - Reduction in repeat vulnerabilities
- Learn from:
  - Incidents
  - Audit findings
  - Past weaknesses
- Continuously adapt testing strategies

References

- Top 5 Security Testing Challenges and Their Solutions
- Continuous Security Assurance | Feuji
- Why AI-Accelerated Development Challenges Traditional Security Testing
- Cybersecurity | Insights, case studies & services | EY - US | EY – US
- Message from SentinelOne
- Technical guide to information security testing and assessment

© 2026 Divya Ranjan, Ph.D. – Optimizing Software Testing Effectiveness